

2. Megatron-Core MoE 架构

本节分两部分介绍 MoE 实现架构：

1. **内部设计**：MoE 层本身的模块化组件（路由器、token 分发器、专家）以及将输入 token 转换为输出表示的四阶段前向传递
2. **外部设计**：并行进程组如何组织以支持分布式执行，以及优化器如何区别处理专家参数

2.1 MoE 层架构与前向传递

MoE 层用多个专家 FFN 替换 Transformer 块中的密集 FFN，每个 token 只有一个子集被处理。Megatron-Core 通过三个模块化组件实现这一架构：

- **路由器 (Router)**：token 到专家的分配
- **Token 分发器 (Token Dispatcher)**：跨 GPU 通信抽象
- **专家 (Experts)**：计算模块

关注点分离的价值：路由器可以融入 CUDA Graphs 而不影响分发器逻辑；分发器可以在 all-to-all 和 DeepEP 之间切换而不修改专家计算；专家实现可以透明地使用不同的 GEMM 后端。

2.1.1 前向传递：路由 → 分发 → 计算 → 合并

MoE 层通过四个顺序阶段处理输入 token：

阶段1：路由 (Route) TopKRouter 确定哪些专家处理每个 token：

- 学习到的线性投影将每个 token 的隐藏状态映射到 E 个 logits（每个专家一个）
- 评分函数（softmax 或 sigmoid）将 logits 转换为概率
- top-k 选择找出每个 token 得分最高的专家

路由器输出两个张量：`probs`（路由权重）和 `routing_map`（布尔掩码，表示 token-专家分配）。为数值稳定性，路由器可通过 `--moe-router-dtype fp32` 在 FP32 下运行。

阶段2：分发 (Dispatch) Token 分发器为跨 GPU 通信准备 token：

- 首先对 token 进行排列，使发往同一专家的 token 连续存储——这对专家侧高效密集 GEMM 至关重要
- 然后使用三种后端之一将 token 移动到承载其分配专家的 GPU：
 - **AllGather**：简单但内存密集，适合小 EP 规模
 - **all-to-all**：标准 NCCL 点对点通信，扩展性好但有同步开销

- **Flex**: 统一设计, 支持 DeepEP 和 HybridEP 优化后端

阶段3: 专家计算 (Expert Computation) 每个 GPU 对接收到的 token 执行本地专家计算。所有本地专家通过 `TEGroupedMLP` 在单次分组 GEMM 调用中运行, 即使单个专家工作负载较小也能最大化 GPU 利用率。

阶段4: 合并 (Combine) 逆向通信将处理后的 token 返回到其原始 GPU, 随后进行反排列以恢复原始序列顺序。如果配置了共享专家, 其输出 (可与路由专家并行计算) 在此阶段被添加。

2.1.2 路由器: Token 到专家的分配

路由器通过两个操作将全局 token 批次转换为专家特定的工作负载:

1. **门控 (Gating)**: 线性投影 $W_r \in \mathbb{R}^{h \times E}$ 将每个 token 的隐藏状态 $x \in \mathbb{R}^h$ 映射到 logits $l = W_r^T \cdot x \in \mathbb{R}^E$
2. **Top-k 选择**: 评分函数将 logits 转换为概率 (softmax 或 sigmoid, DeepSeek-V3 使用 sigmoid), 选择得分最高的 k 个专家

负载均衡策略 (详见第7节):

- 辅助损失 (aux_loss): 可微分, 梯度驱动的软均衡
- Sinkhorn 算法: 基于分配的硬均衡
- 专家偏置 (Expert Bias): 无辅助损失, 基于反馈的自适应均衡

2.1.3 Token 分发器: 通信抽象

分发器通过六阶段流水线管理 GPU 间 token 移动:

```
dispatch_preprocess → token_dispatch → dispatch_postprocess (前向)
combine_preprocess → token_combine → combine_postprocess (反向)
```

三种通信后端:

- **AllGather**: 每个 GPU 收集所有 token 并过滤本地专家。简单但内存密集, 适合小 EP
- **All-To-All**: 标准 NCCL 点对点通信, 每个 GPU 只发送目标节点需要的 token, 扩展性好
- **Flex**: 统一设计, 支持 DeepEP (通过重叠隐藏 NVLink 通信延迟) 和 HybridEP (在 NVL72 等 NVLink 丰富拓扑上提供更高带宽)

2.1.4 专家: 计算模块

每个专家是一个两层 MLP (带可选门控, 用于 SwiGLU/GeGLU 激活)。分组 GEMM 支持多个专家计算的高效批处理。

两种实现:

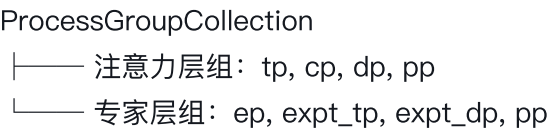
- **TEGroupedMLP**: Transformer Engine 优化实现，支持 FP8 和 FP4 量化
- **SequentialMLP**: 逐一执行专家，适合调试但速度显著较慢

共享专家: DeepSeek-V2/V3、Qwen 等架构包含处理所有 token 的共享专家（不受路由影响）。共享专家计算可与分发-计算-合并流水线并行运行，隐藏其延迟。

2.2 系统集成：并行与优化器

2.2.1 并行组管理

MoE 层需要与密集层不同的进程组，因为不同组件有不同的通信模式。Megatron-Core 通过 `ProcessGroupCollection` 组织这些组：



组件	使用的组	原因
路由器	tp, cp, tp_cp	权重在 EP ranks 间复制
Token 分发器	ep, tp_ep	跨专家 ranks 的 all-to-all
专家	ep, expt_tp, expt_dp	跨 EP 分片；梯度在 EDP 中规约
共享专家	tp	与密集 MLP 相同

这种分离使 Parallel Folding 成为可能：注意力层可使用 TP=4，而 MoE 层使用 ETP=1 配合更高 EP，独立优化每种层类型。

2.2.2 优化器与梯度处理

专家参数在分布式优化中需要特殊处理。Megatron-Core 使用 `ChainedOptimizer` 分别包装密集参数和专家参数的优化器。三个关键设计决策：

1. **参数标识**: 专家参数标记 `allreduce=False`，与使用标准 DP 梯度规约的密集参数区分
2. **独立规约组**: 密集层在 `dp_cp_group` 中规约梯度，专家在 `expt_dp_group` 中规约
3. **梯度缩放**: 专家梯度按 `edp_size / dp_size` 缩放，以补偿专家（处理路由依赖的 token 子集）与密集层之间有效批大小的差异

3. 扩展 MoE：Parallel Folding 与多维并行

3.1 为什么需要并行，MoE 有何不同

3.1.1 大模型训练为何需要并行

内存是根本约束。以 Llama-405B（BF16 + Adam 优化器）为例：

组件	内存需求
模型参数	~810 GB
优化器状态 (Adam)	~4860 GB
梯度	~1620 GB
激活 (8K序列)	~5575 GB
总计	~12865 GB

远超任何单 GPU 容量，多 GPU 不是可选，而是必须。

计算吞吐量是效率原因。单 GPU 计算容量有限，大模型训练需要天文数字级的 FLOPs；聚合多个 GPU 提高吞吐量，将训练时间从数年缩短至数周。

3.1.2 并行的权衡

并行不是免费的，引入的开销包括：通信开销、同步等待、流水线气泡（Pipeline Bubbles）、计算强度降低（分片减小了每 GPU 矩阵规模，降低 GEMM 效率）。

密集模型的关键洞察：并行的"代价"和"收益"成比例增长。更多参数需要更多 GPU，但也意味着每次前向-反向传递更多计算，通信占比保持相对稳定，MFU 较稳定。

MoE 的稀疏性打破了这一平衡：每个 token 只激活 K 个专家 ($K \ll E$)，总参数随 E 增长而每 token 计算只随 K 增长——需要更多 GPU 存储内存，但每 token 计算量不随之增长，通信开销暴露无遗。

3.2 MoE 并行的挑战

3.2.1 MoE 的并行悖论

模型	总参数	激活参数/token	比例
Llama-70B（密集）	70B	70B	1:1
DeepSeek-V3（MoE）	685B	37B	18:1

DeepSeek-V3 的参数是其激活计算所暗示的 18 倍，形成复合效应：

1. 内存快速增长：强制分布到更多 GPU
2. 通信增加：EP 引入的 all-to-all 流量随 K 增长（每个 token 分发到 K 个专家）
3. 计算保持低水平：每 token FLOPs 只随 N_{active} 增长，不足以重叠日益增长的通信

结论：MoE 训练在本质上是通信受限的，除非并行是专门为这种不对称性设计的。

3.2.2 传统并行策略

- 张量并行 (TP)：沿隐藏维度分片权重矩阵，各 GPU 计算部分结果后通过 AllGather/ReduceScatter 合并
- 流水线并行 (PP)：按层将模型分割到 GPU，微批次流经流水线，引入流水线气泡和 P2P 通信
- 数据并行 (DP)：跨 GPU 复制模型，每 GPU 处理不同数据批次，通过 AllReduce 同步梯度
- 上下文并行 (CP)：沿序列维度将输入序列分区到 GPU，长上下文训练的关键

传统并行对 MoE 的失效原因：

- TP 用于 MoE 专家：专家隐藏维度小，TP 导致更小的分片，降低计算效率，放大通信占比
- 仅用 PP：MoE 参数量巨大，需要很多流水线阶段，产生过多气泡
- 仅用 DP：无法分片参数，万亿参数模型不可行

3.2.3 专家并行 (EP)：第五个维度

EP 将专家分布到 GPU 上：EP 度为 E 时，每个 GPU 持有 E/EP 个专家。前向传递：

1. 路由：路由器为每个 token 选择 top-k 专家
2. 分发：all-to-all 通信将 token 发送到持有其分配专家的 GPU
3. 计算：每个 GPU 仅使用其本地专家处理 token
4. 合并：all-to-all 通信将结果返回到原始 GPU

EP 的独特权衡：

- 通信：all-to-all 集合通信，体积随 token 数增长，不随专家数增长
- 计算：每个 GPU 运行更少专家，但每个专家处理其完整隐藏维度
- 内存：更高 EP = 每 GPU 更少专家 = 更低内存压力

EP 的两大关键优势：

1. 将不同 GPU 的 token 分组到单个专家，提高计算强度，改善 GEMM 效率
2. all-to-all 通信体积随专家数增加保持恒定，只有 GPU 数量变化

3.2.4 将 EP 与传统并行结合的挑战

单个 Transformer 块包含两种根本不同的计算模式：

方面	注意力（密集）	MoE（稀疏）
计算	每个 token 关注所有其他	每个 token 路由到 K 个专家
TP	大 QKV 矩阵受益于高 TP	小的每专家维度使高 TP 适得其反
CP	长序列受益于高 CP	无序列依赖，CP 无关
EP	不适用（无专家）	分发众多专家的必需

密集-稀疏失配：传统框架强制两种层类型使用一种并行配置，但它们的最优配置直接冲突：

- 高 TP 有利于注意力但会碎片化专家分片
- 高 CP 有助于长上下文注意力但对 MoE 层毫无收益
- 高 EP 对 MoE 至关重要但对注意力无关

传统方案（ $EP \leq DP$ 约束）引发的三大挑战：

挑战1：乘法 GPU 需求 $EP=8$ 强制 $DP \geq 8$ 。结合 $CP=8$ ，最低需求变为 $1 \times 8 \times 1 \times 8 = 64$ GPU，即使注意力和 MoE 理论上可共享相同的 8 个 GPU。

挑战2：强制次优并行 必须在两种次优配置间选择：高 TP（高效分片注意力矩阵，但碎片化小专家）或低 TP（保留专家计算效率，但注意力层并行度不足）。

挑战3：跨节点通信 EP 约束在 DP 内，高 EP 常迫使 all-to-all 跨节点，带宽比 NVLink 低 5-10 倍。

3.3 Megatron-Core 的解决方案：Parallel Folding 与多维框架

Parallel Folding 是 Megatron-Core 对密集-稀疏失配的解答。**核心思想：**不强迫注意力和 MoE 共享相同的并行配置，让每种层类型独立使用其最优拓扑。

3.3.1 解耦并行映射

Parallel Folding 为注意力和 MoE 层引入独立并行组：

- **注意力层：**在 $TP \times CP \times DP \times PP$ 上形成组，针对序列级密集计算优化
- **MoE 层：**在 $ETP \times EP \times EDP \times PP$ 上形成组，其中 ETP（专家张量并行）和 EDP（专家数据并行）是 MoE 专用维度

唯一约束：PP（流水线并行）必须在两种布局间保持一致，以确保正确的梯度流。

3.3.2 完整的多维并行栈

维度	适用于	目的
TP（张量）	注意力	分片大 QKV/投影矩阵
CP（上下文）	注意力	分发长序列
DP（数据）	注意力	处理不同批次
PP（流水线）	两者	按层分割模型（必须一致）
EP（专家）	MoE	跨 GPU 分发专家
ETP（专家张量）	MoE	在专家内分片（很少使用）
EDP（专家数据）	MoE	为吞吐量复制专家

关键配置原则：

- 注意力层：针对大矩阵（高 TP）和长序列（高 CP）优化
- MoE 层：针对众多小专家（高 EP，通常 ETP=1）优化
- PP：必须在两者间保持一致以确保正确的数据流

3.3.3 Parallel Folding 的优势

Parallel Folding 通过允许 EP 在注意力并行配置的任意子组上"折叠"来消除 $EP \leq DP$ 限制。

示例：注意力配置 TP=4, CP=2, DP=8, PP=4（总 256 GPU）：

- 传统方式： $EP \leq DP = 8$ ，最大 EP 为 8
- 使用 Parallel Folding： MoE 使用 ETP=1, EP=64, EDP=1（同 PP=4）。EP "折叠"跨越 $TP \times CP \times DP$ 组，使专家并行度提高 8x，而注意力层保持最优 TP=4, CP=2

四大优势：

1. 打破 $EP \leq DP$ 约束： EP 现在可以通过在 $TP \times CP$ 组上折叠来超过 DP
2. 降低最低 GPU 需求： CP=8, EP=8 的传统配置需要至少 64 GPU；使用 Folding，CP 和 EP 共享同一 GPU 组，只需 8 GPU
3. 独立优化： 注意力可以为大矩阵使用高 TP，MoE 使用 ETP=1 保持完整专家宽度和更好的 GEMM 效率
4. 保持高带宽通信在 NVLink 域内： CP（注意力）和 EP（MoE）的 all-to-all 通信都可以保持在 NVLink 连接的 GPU 组内，避免较慢的跨节点传输

3.3.4 小结

密集-稀疏失配源于 MoE 的稀疏性：注意力和 MoE 层有冲突的最优并行配置，但传统框架强制它们共享一种配置。**MoE Parallel Folding** 通过解耦并行映射解决了这个问题，允许注意力使用高 TP/CP，MoE 独立使用高 EP。结合分布式优化器和 FSDP，Megatron-Core 将所有并行维度组织成统一框架，将 MoE 训练扩展到数千 GPU。